



Scaling Real-Time Recommendation Systems

From 100 users to millions: Architecting recommendation systems where latency, cost, and accuracy must scale together.

Real-time recommendations have become the competitive moat for industry leaders like TikTok, Netflix, and Amazon.

Why Real-Time Recommendations Matter

Drive Engagement

Personalized feeds keep users active and scrolling longer. Users expect content tailored to their exact interests and behaviors in the moment.

Increase Revenue

Higher CTR, upsell opportunities, and conversion rates directly impact the bottom line. Real-time systems can generate 30-45% higher conversion rates.

Reduce Churn

Maintain user interest by serving fresh, relevant content that adapts to shifting preferences instantly.



The Scale Challenge: Serving **billions of requests daily** while maintaining sub-100ms latency.

Core Components of RecSys Infrastructure

1

Event Streaming

Captures and processes user interactions (clicks, views, purchases) in real-time through Kafka, Kinesis, or Pulsar pipelines.

2

Feature Store

Maintains consistency between real-time and offline features, ensuring ML models get identical inputs during training and serving.

3

Vector DB / ANN Index

Enables efficient similarity search at scale across billions of high-dimensional embeddings representing users and items.

4

Model Inference

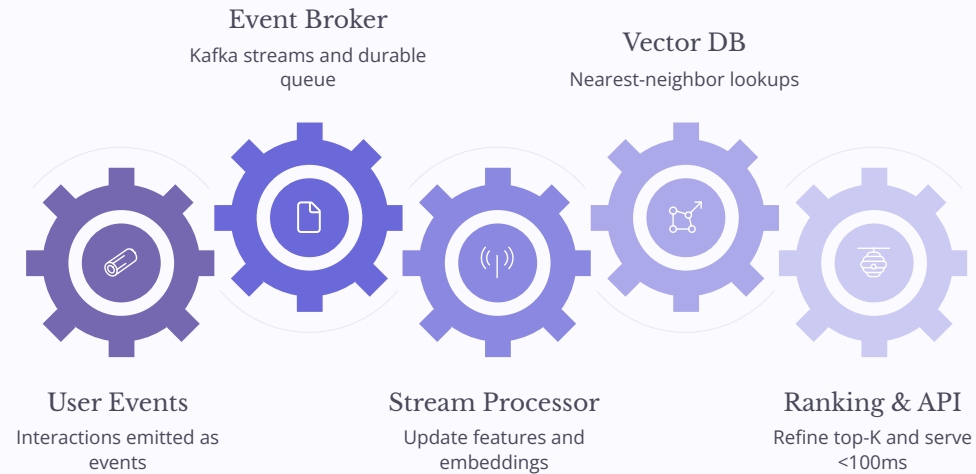
Executes ranking, embedding generation, and deep neural networks to score and sort recommendation candidates.

5

API Serving

Delivers recommendations with low latency and high throughput, often through distributed microservices architecture.

Data Flow in Real-Time RecSys



The critical path for real-time recommendations requires orchestrating multiple systems to deliver relevant content in milliseconds:

1. User interactions trigger events captured by instrumentation
2. Event brokers (Kafka) enable reliable, ordered delivery
3. Stream processors update feature values and embedding vectors
4. Vector search identifies candidate items for ranking
5. ML models refine and score final recommendations
6. API layer delivers results to front-end applications

Scaling Challenge #1: Feature Freshness

The Problem

Stale features = irrelevant recommendations

When user behavior changes rapidly (e.g., shopping for a gift), recommendations based on outdated signals deliver poor experiences.

The Solution

Implement **streaming updates** to feature store platforms like Feast, Tecton, or Hopsworks that maintain consistency across:

- Training (offline batch) datasets
- Inference (online serving) feature values



⚠ Challenge: Different latency SLAs for different features (milliseconds for clicks, seconds for cart adds, minutes for purchases)

Scaling Challenge #2: Vector Search at Scale

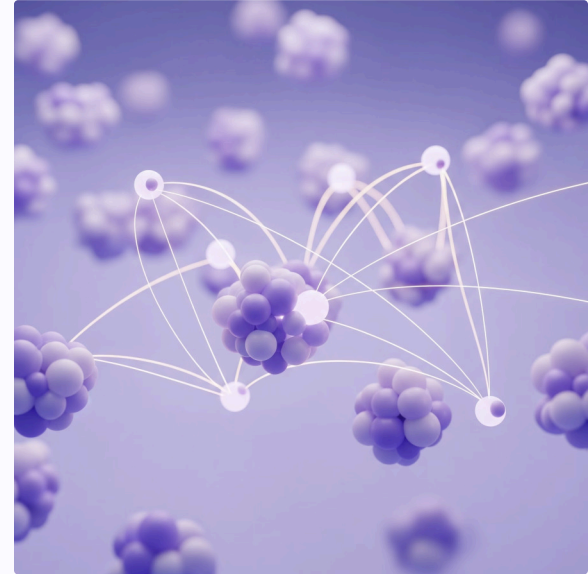
Modern recommendation systems rely on embedding vectors to represent users and items in high-dimensional space (64-1024 dimensions).

The Challenge

Searching through **billions of embeddings** for nearest neighbors in sub-10ms is computationally expensive.

Solution: Approximate Nearest Neighbor (ANN)

- Trade perfect recall for dramatic speed improvements
- Technologies: FAISS, Pinecone, Weaviate, Milvus, Vespa
- Techniques: HNSW, IVF, PQ quantization, clustering



❏ Key tradeoff: Accuracy vs. Latency vs. Memory

Quantization can reduce memory by 4-8x with minimal accuracy loss

Scaling Challenge #3: Model Inference



Inference often dominates end-to-end latency budget

The Challenge

Ranking models (DNNs, Transformers) are computationally intensive for real-time inference at scale.

Optimization Techniques

- **Model Optimization:** Quantization (INT8/FP16), knowledge distillation, pruning, TensorRT/ONNX conversion
- **Microbatching:** Group requests to balance throughput and latency (1-8ms batching windows)
- **Hardware Acceleration:** GPUs, AWS Inferentia, Google TPUs, Habana Gaudi

Scaling Challenge #4: API Throughput

Global Load Distribution

Multi-region deployment with intelligent routing to handle millions of RPS worldwide.

- Edge computing for low-latency global serving
- CDN integration for cacheable components
- Geographic routing based on user location

Caching Strategies

Reduce computational load by caching "popular" recommendations.

- User-segment based caching (cohorts)
- TTL based on content velocity
- Cache invalidation on feature updates

Safe Deployment

Minimize risk when deploying new recommendation models.

- Canary deployments (1% → 10% → 100%)
- Shadow testing against production traffic
- Automated rollbacks on metric degradation



Monitoring at Scale

Key Metrics

Business Metrics

- Click-Through Rate (CTR)
- Conversion Rate
- Revenue per Session
- Session Duration

System Metrics

- Latency (P95/P99)
- Throughput (RPS)
- Error Rate
- Cost per 1K Recommendations

ML Metrics

- Recall@K, Precision@K
- NDCG (Normalized Discounted Cumulative Gain)
- Feature/Embedding Drift
- Diversity & Coverage



A/B Testing Infrastructure

Continuous experimentation requires:

- **Reliable user assignment** to test/control groups
- Statistical analysis tooling for experiment results
- Guardrail metrics to prevent negative business impact

Case Study: TikTok-Style Feed

Problem

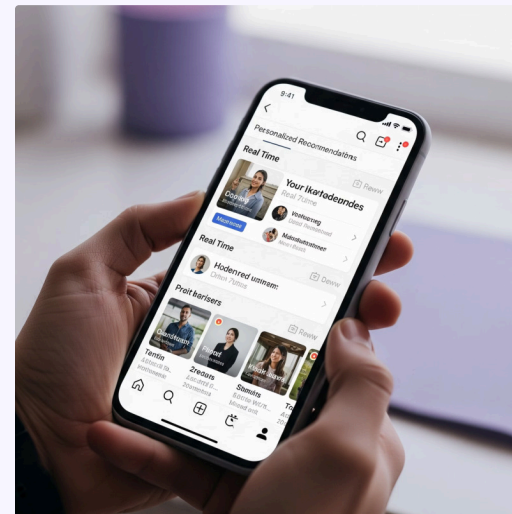
Content feed becoming irrelevant at scale, especially for new users with limited history.

Solution

Streaming architecture capturing real-time signals to update user embeddings and content relevance scores.

Infrastructure

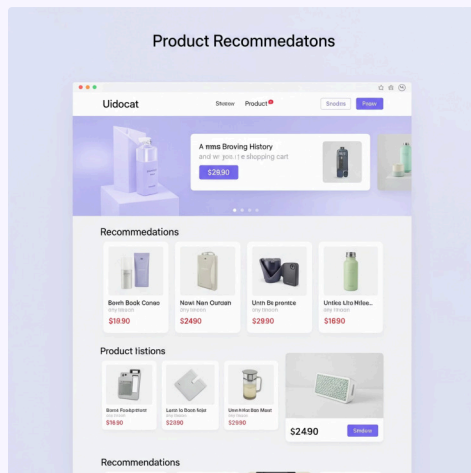
- **Kafka**: Event streaming for user interactions
- **Flink**: Stream processing for feature updates
- **FAISS**: Vector similarity search for content
- **TensorRT**: Optimized DNN inference for ranking



✓ Results

- Engagement increased 20%
- Average session time +35%
- End-to-end latency reduced to 80ms
- New user retention improved 15%

Case Study: Amazon-Style E-Commerce



✓ Results

- 2× conversion rate
- 30% reduction in infrastructure costs
- 15% increase in average order value
- Processing 15M+ recommendations per minute

Problem

Scaling product recommendations across a global catalog with millions of SKUs while maintaining relevance and freshness.

Solution

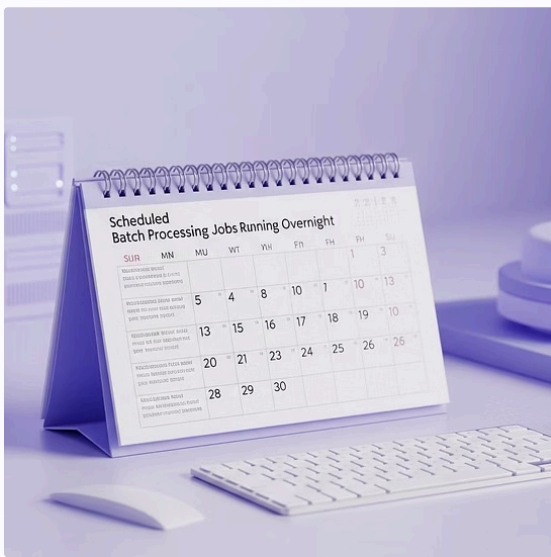
Hybrid architecture combining:

- **Batch computation**: Offline precomputed base recommendations
- **Real-time refresh**: Contextual signals update rankings

Infrastructure

- **Spark**: Batch computation of item similarities
- **Tecton**: Feature store for training/serving consistency
- **Pinecone**: Vector DB for similarity search
- **KServe**: Model serving with autoscaling

Real-Time vs Batch Trade-Off



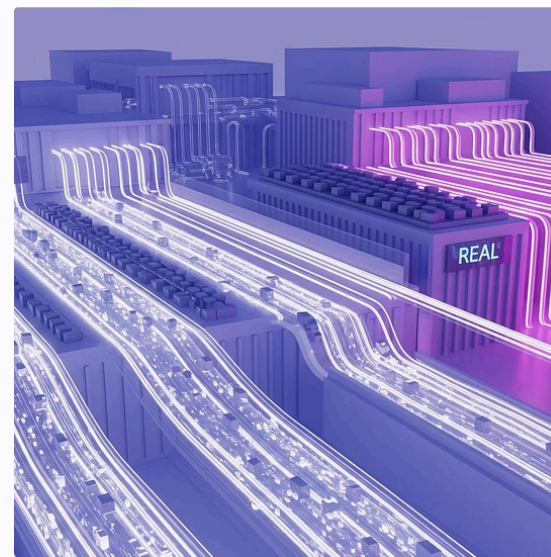
Batch Processing

- **Cost:** Lower compute and storage requirements
- **Freshness:** Daily/weekly updates
- **Complexity:** Simpler architecture
- **Scale:** Easier to scale with fixed time windows



Real-Time Processing

- **Cost:** Higher due to always-on processing
- **Freshness:** Millisecond-level updates
- **Complexity:** More complex streaming architecture
- **Scale:** Challenging to scale elastically



Hybrid Approach

The Best Solution: Precompute baseline recommendations in batch, then refresh and re-rank with real-time signals.

Gradually shift more computation to real-time as user base and engagement grow.

Real-Time Recommendation Architecture



Scaling RecSys = Streaming + Embeddings + Ranking

The most effective architectures combine these three elements with careful orchestration of data flows and processing stages.

Summary: Scaling Real-Time Recommendation Systems

Key Components for Scale

Successfully scaling recommendation systems requires three integrated systems:

- **Streaming infrastructure** for real-time event processing
- **Vector databases** for efficient similarity search
- **Optimized inference** for fast, cost-effective scoring

Primary Challenges

- Maintaining feature freshness without breaking the budget
- Balancing latency requirements with accuracy needs
- Handling throughput spikes without over-provisioning
- Managing infrastructure costs as scale increases

Enterprise Differentiator

Leading companies treat recommendation systems as core strategic assets that drive competitive advantage.



Next Steps: Start with hybrid approaches and incrementally shift more to real-time as you validate ROI.